Feature List Categorized

WebAdvisor Clone - Week-by-Week Feature List

Organized by Design Pattern

Week 1 — Singleton Pattern

There must be exactly one shared resource managing all database access for the entire application.

- The application maintains a single, shared database connection that all features use — it is never created more than once, no matter how many parts of the system request it
- On first startup, the database automatically creates all necessary tables if they do not already exist
- A pool of reusable database connections is managed centrally — connections are borrowed and returned rather than opened and closed repeatedly
- If a database connection fails, the system automatically retries before reporting an error
- A single shared configuration manager loads application settings (database URL, pool size, app mode) once from a properties file and serves them to any part of the system that needs them
- A single shared logger writes all application events to both the console and a daily log file — every component uses the same logger instance

- The application can be run in Development mode or Production mode, controlled by a single configuration value read once at startup

---

Week 2 — Factory Method Pattern

The system needs to create different types of users (Student, Faculty, Admin) without the calling code needing to know which specific type is being built.

- A new Student account can be created by providing the required information — the system produces a fully initialized Student object with all student-specific fields populated
- A new Faculty account can be created by providing the required information — the system produces a fully initialized Faculty object with all faculty-specific fields populated
- When creating any user, the system validates that required fields are present, the username is unique, the email address is properly formatted, and the username meets length and character rules — before the object is created
- A user can be looked up by username and the system automatically returns the correct type of user object (Student or Faculty) without the caller having to specify which type to expect
- All active students can be listed with their basic profile information
- All active faculty can be listed with their basic profile information
- A student's profile summary and a faculty member's profile summary display different information appropriate to each type, using the same method call on either object

---

Week 3 — Strategy Pattern

The way a user is authenticated can change depending on the environment or security requirements, without changing any of the code that triggers login.

- **Login** — A user enters their username and password; the system verifies their identity using whatever authentication method is currently active
- **Development Mode Login** — In development/testing, authentication compares passwords directly with no hashing (a clearly flagged, unsafe-for-production method)
- **Standard Secure Login** — In production, authentication hashes the entered password with a stored salt and compares the result to the stored hash
- **Two-Factor Login** — An enhanced login that first verifies the password, then generates a time-limited one-time code (simulated delivery via console/log), and requires the user to enter the code to complete login

- **Switch Authentication Method** — An administrator can change which authentication method the application uses at runtime without restarting the system
- **What's My User ID?** — A user provides their first name, last name, and email address; the system looks up and returns their username
- **Forgot Password / Reset Password** — A user provides their username and email; the system generates a temporary reset token (simulated delivery), the user enters the token, and is then allowed to set a new password
- **Change Password** — An authenticated user enters their current password (verified by the active authentication method), then enters and confirms a new password
- **Password Strength Check** — When setting a new password, the system evaluates it against rules (length, uppercase, lowercase, digits, special characters) and gives the user feedback before accepting it
- **Logout** — The user's authenticated session is ended and they are returned to the login screen; the event is recorded in the audit log

---

Week 4 — Observer Pattern

When something important happens in the system — a grade is posted, a payment is confirmed, a hold is placed — every interested party should be automatically informed without the event source needing to know who is listening.

- **In-App Notification Inbox** — Students and Faculty each have a personal inbox where they receive notifications from any part of the system; unread notifications are counted and displayed
- **Mark Notifications as Read** — A user can mark individual notifications as read, or mark all notifications as read at once
- **Filter Notifications** — A user can filter their inbox by notification type, urgency level, or date range
- **Grade Posted Alert** — When a grade is submitted for a student, that student is automatically notified without the grading feature needing to directly contact the student
- **Registration Confirmation Alert** — When a student's enrollment status changes (registered, dropped, moved to waitlist, promoted off waitlist), they are automatically notified
- **Financial Aid Update Alert** — When a student's financial aid status changes (award posted, disbursement processed, action required), they are automatically notified
- **Payment Confirmation Alert** — After a successful payment is processed, the student is automatically notified with the amount and new balance

- **Restriction Placed Alert** — When any hold is added to a student's account, they are automatically notified with the type of hold and the office to contact
- **Restriction Removed Alert** — When a hold is cleared from a student's account, they are automatically notified
- **Simulated Email Delivery** — When a notification is triggered, a simulated email is generated and logged (respecting the user's email notification preference)
- **Simulated SMS Delivery** — When a notification is triggered, a simulated SMS is generated and logged (respecting the user's SMS notification preference)
- **Notification Preferences** — A user can turn email, SMS, and in-app notifications on or off independently, and can enable or disable specific notification types; preferences are saved and respected by all future notifications
- **All Notifications Logged to Audit Trail** — Every notification event is written to the system audit log regardless of user delivery preferences

---

Week 5 — Command Pattern

Every significant action a user takes (registering, dropping, paying, updating a profile) should be captured as a self-contained object that can be executed, recorded, and — where appropriate — undone.

- **Register for a Section** — A student's registration action is captured and executed as a discrete unit; if something goes wrong, the entire action is rolled back cleanly
- **Drop a Section** — A student's drop request is captured and executed as a discrete unit; the action is reversible within an allowed window
- **Withdraw from a Section** — A student's withdrawal after the census date is captured as a distinct action from a drop, with different consequences (W grade); this action is not undoable
- **Add to Waitlist** — Adding a student to a section waitlist is captured as an action, including their position in line
- **Remove from Waitlist** — Removing a student from the waitlist (either by their choice or by promotion) is captured as a distinct action
- **Make a Payment** — A payment transaction is captured as a complete action including amount, method, and account details; the action can be reversed within 24 hours
- **Set Up a Payment Plan** — Creating an installment payment plan is captured as a single action; it can be undone if no payments have been made yet
- **Update Contact Information** — A user's changes to their phone, address, or emergency

contact are captured so that previous values can be restored if needed

- **Update Email Address** — A user's email change is captured as a reversible action
- **Update Faculty Office Hours** — A faculty member's office hours change is captured and fully reversible
- **Submit a Grade** (Faculty) — A faculty member's grade entry for a student is captured as an action; it can be corrected until grades are finalized
- **Finalize Grades** (Faculty) — The decision to lock all grades for a section is captured as a permanent, non-reversible action
- **Undo Last Action** — A user can undo their most recent reversible action; the system displays what will be undone before the user confirms
- **Redo Last Action** — A user can re-execute the most recently undone action
- **View My Transaction / Action History** — A user can view a chronological list of all actions they have taken, with timestamps and descriptions, filterable by type and date
- **Batch Register for Multiple Sections** — A student selects several sections and registers for all of them as a single all-or-nothing operation; if any one fails, none are registered

---

Week 6 — State Pattern

The registration system behaves completely differently depending on where in the academic calendar it currently sits — and a section enrollment record also progresses through a defined sequence of statuses.

- **Registration Not Yet Open** — Before the registration window opens, any attempt to register or join a waitlist is blocked with a message telling the student when registration opens
- **Registration Open** — During the registration window, students can register for open sections or join waitlists for full sections; all registration features are available
- **Section Full / Waitlist Mode** — When a section has no available seats, new registration requests are automatically routed to the waitlist; existing enrollments can still be dropped
- **Registration Closed (Before Census)** — After the registration window closes but before the census date, new registrations and waitlist additions are blocked; drops are still allowed
- **After Census Date** — After the census date, standard drops are no longer available; students may only withdraw (which results in a W grade on the transcript)
- **After Withdrawal Deadline** — After the last day to withdraw, no student-initiated enrollment changes are allowed; only administrative changes are possible
- **Registration Date & Time Lookup** — A student can view the current registration state, what actions are available to them right now, and the dates of all upcoming state transitions

(open date, close date, census date, withdrawal deadline)

- **Enrollment Status Tracking** — Each enrollment record progresses through its own sequence of statuses (Registered → Dropped, Registered → Withdrawn, Waitlisted → Promoted, etc.); transitions that would skip or reverse the sequence are blocked
- **Official Transcript Request Status** — A transcript request progresses through defined stages (Pending → Processing → Ready → Shipped → Completed, or Cancelled); each stage transition notifies the student automatically
- **Admin: Advance Registration State** — An administrator can manually move the system from one registration state to the next (e.g., open registration, close registration, pass census date)

Week 7 — Decorator Pattern

Users need different levels of access to system features, and those access levels need to change dynamically — for example when a hold is placed or when a student qualifies for additional privileges — without modifying the user objects themselves.

- **Student Feature Access** — A student account automatically gains access to registration, grade viewing, transcript ordering, financial features, and financial aid — and nothing that belongs to faculty
- **Faculty Feature Access** — A faculty account automatically gains access to class rosters, grade submission, section search, budget features, and student profile viewing — and nothing that belongs to students
- **Admin Feature Access** — An admin account gains access to all student and faculty features plus system management tools
- **Honors Student Additional Access** — An honors student can be given priority registration access and access to honors-designated sections on top of their standard student permissions, without modifying the student account itself
- **Athlete Additional Access** — A student athlete can be given priority registration access and access to athletic advising features on top of their standard student permissions
- **Financial Hold Applied** — When the Bursar places a financial hold on a student, that student loses the ability to register for classes and order official transcripts until the hold is removed; all other access remains
- **Academic Hold Applied** — When an academic hold is placed, the student loses the ability to register for classes; they can still view grades and transcripts
- **Library Hold Applied** — When a library hold is placed, the student loses the ability to order

official transcripts

- **Hold Removed** — When any hold is cleared, the permissions that were removed are immediately restored without any other changes to the account
- **View My Restrictions** — A student can view a list of all active holds on their account, including the type of hold, a description, the office that placed it, and instructions for resolving it
- **Access Denied Message** — When a student tries to access a feature they are blocked from, the system explains specifically which restriction is preventing access and what they need to do to resolve it
- **Feature Access Check Before Every Action** — Before any significant feature executes, the system verifies the user has permission; this check happens automatically without the feature itself needing to contain permission logic

---

Week 8 — Template Method Pattern

Every report in the system follows the same overall process — verify access, gather data, format content, add standard headers and footers, produce output, and log the generation — but the specific data gathered and the format produced are different for every report type.

- **View Unofficial Transcript** — A student views their complete academic history organized by semester, including course codes, course names, credits, grades, semester GPA, and cumulative GPA; the document is clearly watermarked as unofficial
- **Order Official Transcript** — A student requests an official transcript, specifies the delivery method (electronic or physical mail) and recipient; the generated document includes institutional header, registrar certification line, and issue date; no unofficial watermark
- **Student Finance Summary** — A student views a complete summary of their account: all charges by term (tuition, fees), all payments made, pending financial aid, and the current balance due
- **Student Tax Document (1098-T equivalent)** — A student views or downloads their annual tuition payment summary for tax purposes, showing amounts paid and scholarships received for a selected calendar year; watermarked with a tax advice disclaimer
- **Payment Receipt** — Immediately after a payment is processed, the student receives a formatted receipt showing transaction ID, date, amount, payment method, and updated balance
- **Class Roster** (Faculty) — A faculty member generates a formatted list of all enrolled students for a selected section, including student name, ID, email, major, and enrollment date;

includes a summary line showing total enrolled and seats remaining

- **Census Roster** (Faculty) — A faculty member generates a version of the class roster locked to the census date snapshot, with an attendance/participation status column added; once generated this document is versioned and cannot be altered
- **Gradebook Report** (Faculty) — A faculty member views all students in a section alongside their submitted grades, with class average, median, and grade distribution calculated automatically
- **Budget Summary Report** (Faculty) — A faculty member views their department's budget broken down by category, showing allocated amounts, amounts spent year-to-date, and remaining balance per category

---

Week 9 — Facade Pattern

The underlying system is now made up of many interacting patterns and subsystems. The user interface should not need to coordinate all of them directly — a single simplified entry point should handle all of that complexity invisibly.

- **Register for a Course (one call)** — The UI calls a single registration method; behind the scenes, all prerequisite checks, schedule conflict detection, seat availability checks, financial hold checks, credit limit checks, and waitlist routing happen automatically and in the correct order; the UI receives a single typed result (SUCCESS, WAITLISTED, PREREQ_NOT_MET, etc.) and displays the appropriate message
- **Drop a Course (one call)** — The UI calls a single drop method; the correct action (drop vs. withdraw) is determined automatically based on the current academic calendar state; the waitlist is promoted if applicable; the student is notified automatically
- **Get Current Registration Status** — The UI calls a single method and receives everything needed to display the registration page: current window state, what actions are available, all relevant dates, and the student's current schedule
- **Program Evaluation (one call)** — The UI calls a single method; behind the scenes, the student's completed courses are matched against their degree requirements, in-progress courses are considered, remaining requirements are identified, and next-semester course suggestions are generated; the UI receives a single structured result to display
- **My Educational Plan (one call)** — The UI calls a single method and receives a semester-by-semester course plan organized around remaining degree requirements
- **Check Graduation Eligibility (one call)** — The UI calls a single method and receives either a confirmation of eligibility or a specific list of unmet requirements

- **Get Financial Summary (one call)** — The UI calls a single method and receives a complete financial picture: charges, payments, financial aid, payment plan status, and balance due — aggregated from multiple sources
- **Make a Payment (one call)** — The UI calls a single method; payment validation, external payment processing, account update, receipt generation, and student notification all happen automatically
- **Submit Grade (one call)** (Faculty) — The UI calls a single method per student grade; all validation (grading window, valid grade value, faculty authorization) happens automatically; the student is notified automatically on success
- **Finalize Section Grades (one call)** (Faculty) — The UI calls a single method; the system verifies all students have grades, locks the gradebook, and confirms completion

---

Week 10 — Adapter Pattern

The system needs to work with external services and legacy systems that have their own incompatible interfaces — the rest of the application should not know or care that an external system is involved.

- **NBS Payment Processing** — When a student makes a payment, the system communicates with the simulated NBS external payment service; the rest of the application simply calls the standard internal payment interface and never knows it is talking to an external system
- **NBS Payment Plan** — When a student sets up a payment plan, the simulated NBS installment service is called through a standard internal interface; the resulting schedule is returned in the application's own data format
- **NSC Self-Service Link** — When a student visits the NSC Self-Service page, the system retrieves their enrollment verification data from the simulated National Student Clearinghouse external API and presents it in the application's own format; a properly formatted deep-link URL to the NSC portal is also generated
- **Legacy Student Finance Data** — Historical financial records stored in a simulated legacy finance system (with its own data formats and student ID scheme) are retrieved and converted into the application's standard financial data format, then displayed alongside current system data
- **Email Delivery** — When the notification system needs to send an email, it calls a standard internal notification interface; the actual call to the simulated external email gateway (with its own parameter format and response structure) is hidden behind that interface
- **SMS Delivery** — When the notification system needs to send an SMS, it calls the same

standard internal notification interface; the simulated external SMS gateway's specific format and character limits are handled invisibly

- **Third-Party Transcript Fulfillment** — When an official transcript is ordered for physical delivery, the request is routed to a simulated external transcript fulfillment service through a standard internal interface; the external service's response is converted back into a standard transcript request status update

---

Week 11 — Composite Pattern

The academic structure of the institution — and a department's budget — are both naturally hierarchical: colleges contain departments, departments contain programs, programs contain courses, and courses contain sections. Individual items and groups of items should be usable in the same way.

- **Schedule of Classes** — The course catalog is organized and displayed as a hierarchy (College Department Course Section); a user can expand and collapse any level; credits and section counts roll up automatically to each parent level
- **Course Search within the Catalog** — A user can search the entire catalog tree by course code or keyword and receive matching sections regardless of where they sit in the hierarchy
- **Program Requirements Tree** — A student's degree requirements are organized as a hierarchy of requirement groups, sub-groups, and individual courses; credit totals and completion percentages roll up automatically through every level of the tree
- **My Educational Plan** — A student's multi-semester course plan is organized as a hierarchy of planned semesters containing planned courses; total planned credits per semester are calculated automatically
- **Prerequisite Chain Display** — A course's prerequisites (which may themselves have prerequisites) are displayed as a nested tree showing the full chain of requirements
- **Section Detail Leaf** — An individual course section (a leaf in the catalog tree) displays its specific details: instructor, meeting days and times, room, building, total seats, enrolled count, waitlist count, and delivery mode
- **Budget Category Hierarchy** (Faculty) — A department's budget is organized as a hierarchy of budget categories containing individual line items; allocated amounts, spent amounts, and remaining balances roll up automatically through each level
- **Budget Summary Display** (Faculty) — The entire department budget tree is displayed with each level showing its aggregated totals, allowing a faculty member to see both the big picture and the specific line items in one unified view

---

Week 12 — Iterator Pattern

The system has many different collections — student schedules, class rosters, waitlists, grade lists, section search results — and each collection needs to be traversed in multiple different orders without exposing how the data is stored internally.

- **My Class Schedule — Weekly View** — A student's enrolled sections are walked in day-and-time order (Monday through Friday, then by start time) to produce a weekly schedule grid
- **My Class Schedule — By Semester** — A student's full enrollment history is walked semester by semester; each semester group is presented together
- **My Class Schedule — By Delivery Mode** — A student can filter their schedule to walk only online sections, only in-person sections, or only hybrid sections
- **My Class Schedule — By Status** — A student can walk their enrollments filtered to only active, only dropped, or only withdrawn records
- **Class Roster — Alphabetical** (Faculty) — A faculty member walks the roster in alphabetical order by student last name
- **Class Roster — By Student ID** (Faculty) — A faculty member walks the roster in student ID order
- **Class Roster — By Credits Earned** (Faculty) — A faculty member walks the roster with students who have the fewest credits earned first (useful for identifying students who may need additional support)
- **Class Roster — By Enrollment Date** (Faculty) — A faculty member walks the roster in the order students registered, showing who enrolled first
- **Gradebook — Missing Grades View** (Faculty) — A faculty member walks only the students in a section who do not yet have a grade submitted, making it easy to identify who still needs grading
- **Gradebook — By Grade** (Faculty) — A faculty member walks the grade list sorted from highest to lowest grade
- **Waitlist — By Priority Order** — The waitlist for a section is walked in position order (first in line first), used both for display and for automatic promotion when a seat opens
- **Waitlist — Permission to Add** (Faculty) — A faculty member walks only the waitlisted students they have approved for a permission-to-add, separately from the general waitlist
- **Section Search Results — By Availability** — Search results are walked with sections that have open seats surfaced first
- **Section Search Results — By Instructor** — Search results are walked in alphabetical order

by instructor last name

- **Section Search Results — Online First** — Search results are walked with online sections listed before in-person sections
- **Schedule Conflict Detection** — A student's current schedule and a proposed new section are walked together simultaneously to identify any day-and-time overlaps

---

Week 13 — Abstract Factory Pattern

Students and faculty need entirely different sets of interface components — different dashboards, different menus, different report views — and all components for a given user type must be visually and functionally consistent with each other.

- **Student Dashboard** — When a student logs in, a complete dashboard is assembled: a widget showing their upcoming class schedule, a widget showing their current GPA and credits earned, a widget showing their current account balance and next payment due date, and an unread notification count
- **Faculty Dashboard** — When a faculty member logs in, a completely different dashboard is assembled: a widget showing today's class sections, a widget showing which sections still have unsubmitted grades, a widget showing unread notifications, and a budget remaining widget
- **Student Navigation Menu** — A student's navigation menu contains exactly the items relevant to students: My Schedule, Register for Classes, Grades, Financial Information, Financial Aid, Transcript, Program Evaluation, My Educational Plan, Documents, Restrictions
- **Faculty Navigation Menu** — A faculty member's navigation menu contains exactly the items relevant to faculty: My Schedule, Class Roster, Submit Grades, Section Search, Student Profiles, Permission to Add, Budget Summary, Documents
- **Admin Navigation Menu** — An administrator's navigation menu contains all student and faculty items plus administrative tools: Manage Users, Audit Log, Advance Registration State, Manage Restrictions, System Configuration
- **Student Notification Panel** — The notification panel for a student prominently surfaces grade alerts, registration confirmations, and financial aid updates
- **Faculty Notification Panel** — The notification panel for a faculty member prominently surfaces grade submission deadline alerts, roster change notifications, and budget alerts
- **Student Report Viewer** — The report viewer available to a student shows: unofficial transcript, official transcript request status, tax document, finance summary, and payment receipts; faculty-only reports are not present

- **Faculty Report Viewer** — The report viewer available to a faculty member shows: class roster, census roster, gradebook, and budget summary; student-only reports are not present
- **Student Profile Panel** — The profile panel for a student displays: name, student ID, declared major, GPA, credits earned, advisor name, and enrollment status; the editable fields are phone, address, and email
- **Faculty Profile Panel** — The profile panel for a faculty member displays: name, faculty ID, department, title, office location, and office hours; the editable fields are phone, email, and office hours
- **Student Quick Actions Panel** — A panel of large shortcut buttons for the most common student tasks: Register for Classes, Pay Balance, View Grades, Order Transcript
- **Faculty Quick Actions Panel** — A panel of large shortcut buttons for the most common faculty tasks: View Roster, Submit Grades, View Schedule, Budget Summary
- **Report Output Format Family — PDF** — When a user requests PDF output, all components of the output (renderer, formatter, exporter) are produced as a consistent PDF-producing family
- **Report Output Format Family — HTML** — When a user requests on-screen output, all components are produced as a consistent HTML-rendering family
- **Report Output Format Family — Excel** — When a faculty member exports a roster, all components are produced as a consistent spreadsheet-producing family (for use with external tools such as an LMS)

Week 14 — Chain of Responsibility Pattern

Every major request — registering for a class, submitting a grade, processing a payment — must pass through a series of independent validation checks in a defined order. Each check is responsible for exactly one concern, and any check can stop the entire request with a specific, actionable error message.

Registration Validation Pipeline

The following checks run in order every time a student attempts to register for a section. Each check is independent and knows nothing about the others.

1. **Authentication Check** — Is this user actually logged in? If not, stop immediately and prompt them to log in again
2. **Registration Window Check** — Is registration currently open? If not, stop and tell the student when it will open

3. **Permission Check** — Does this student have registration permission, or does an active hold block them? If blocked, name the specific hold and where to go to resolve it
4. **Prerequisite Check** — Has the student completed all prerequisites for this course? If not, list exactly which prerequisites are missing
5. **Corequisite Check** — If this course requires a corequisite, is the student also enrolled in it (or already completed it)? If not, name the required corequisite
6. **Schedule Conflict Check** — Does the new section's meeting time overlap with any section the student is already enrolled in? If so, name the conflicting section
7. **Credit Limit Check** — Would adding this section push the student over the semester credit maximum? If so, tell them their current total and how to request an override from their advisor
8. **Financial Hold Check** — Does the student have a financial hold that blocks registration? If so, name the hold and direct them to the Bursar's Office
9. **Academic Hold Check** — Does the student have an academic hold that blocks registration? If so, direct them to the appropriate office
10. **Duplicate Enrollment Check** — Is the student already enrolled in a different section of this same course? If so, inform them they are already registered for this course
11. **Seat Availability Check** — Are seats available in the requested section? If the section is full, do not fail — instead flag the request for automatic waitlist routing

Grade Submission Pipeline

The following checks run in order every time a faculty member submits a grade.

1. **Authentication Check** — Is this faculty member logged in?
2. **Permission Check** — Does this faculty member have grade submission permission?
3. **Section Ownership Check** — Is this faculty member actually assigned to teach the section they are grading?
4. **Grading Window Check** — Is the grading period currently open? If not, tell the faculty member when the deadline was or when it opens
5. **Valid Grade Value Check** — Is the submitted grade a recognized grade symbol (A, B, C, D, F, W, I, P, NP, etc.)? If not, list the valid options
6. **Grades Not Finalized Check** — Have grades for this section already been finalized? If so, block any further changes

Payment Processing Pipeline

The following checks run in order every time a student attempts to make a payment.

1. **Authentication Check** — Is this user logged in?
2. **Payment Permission Check** — Does this user have permission to make payments?
3. **Positive Amount Check** — Is the payment amount greater than zero?
4. **Amount Reasonableness Check** — Does the payment amount exceed the current balance due? If so, warn the student before proceeding
5. **Payment Method Validation Check** — If paying by credit card, is the card number format valid, is the expiration date in the future, and is the CVV the correct length?
6. **Daily Limit Check** — Would this payment exceed the maximum allowed daily payment total (fraud prevention)? If so, direct the student to contact the Bursar's Office
7. **External Gateway Availability Check** — Is the external payment service reachable right now? If not, inform the student the payment system is temporarily unavailable and suggest they try again shortly

Cross-Pipeline Features

- **Specific, Actionable Error Messages** — Because each handler is responsible for exactly one check, every failure message tells the student or faculty member precisely what went wrong and exactly what they need to do to resolve it — never a generic error message
- **Pipeline Audit Log** — Every request that passes through any pipeline is logged: which handlers it passed, which handler (if any) it failed at, the timestamp, and the user who made the request; this log is viewable by administrators
- **Request Passes All Checks** — When a request clears every handler in the pipeline, the appropriate command (register, grade submission, payment) executes automatically as the final step

---

System-Wide Features (Present Throughout All Weeks)

- **Audit Log** — Every login, logout, failed login attempt, command execution, undo, report generation, and pipeline result is written to a central audit log with timestamp and user ID
- **Account Lockout** — After five consecutive failed login attempts, a user account is automatically locked and must be unlocked by an administrator
- **Session Management** — A session token is created when a user logs in and invalidated when they log out; accessing any feature without a valid session redirects to login
- **Transactional Data Integrity** — Any operation that writes to the database either completes fully or rolls back completely — partial writes never occur
- **Access Denied Handling** — When a user tries to reach a feature they are not permitted to

access, they receive a clear explanation of why and what to do — never a blank screen or generic error

- **Development Seed Data** — The application ships with a script that populates the database with sample students, faculty, departments, courses, and sections for use during development and testing